

Data, Computation & Innovation II

Instructor: Jared Whalen

Housekeeping

- Chart discussions
- Weekly material/handouts
- Class recordings

Critique (~10 minutes)

Break into groups of ~3 and discuss last week's assignment

- **Author:** Explain what you did and why
- **Editor:** Give specific questions, give specific feedback
 - what's working?
 - what's unclear?
- **Everyone:** What edits would resolve these tensions?

✨ Review ✨

- What are the different data types we discussed in week 1?
- What are the four “layers” in the grammar of graphics that we are focusing on this class?
- You declare `let count = $state(0)` and display it in your template. A button increments it. Your teammate writes the same thing but uses `let count = 0` instead. What's the difference in behavior, and why?
- You're building a chart that should show a "no data" message when the dataset is empty and the chart when it isn't.

Course Overview

Week 1 (June 3): Chart Fundamentals + SVG/Svelte Introduction

Week 2 (June 10): D3 Fundamentals, Part 1

Week 3 (June 17): D3 Fundamentals, Part 2

Week 4 (June 24): Interactivity and animation + *Guest speaker Kavya Beheraj/The Athletic*

Week 5 (July 1): GIS and Mapping + *Guest speaker Alvin Chang/The Pudding*

Week 6 (July 8): Scrollytelling + Advanced patterns

Week 7 (July 15): Final project presentations

D3 Fundamentals, Part 1

Scales and Generators

Introducing D3

D3 stands for Data-Driven Documents. It's a JavaScript library that's been the foundation of interactive journalism graphics for over a decade. If you've seen a sophisticated interactive chart in the NYT, The Pudding, or Reuters, there's a good chance D3 was involved.

At its core, D3 is a collection of small, focused utilities. There's a module for scales, one for shapes, one for geographic projections, one for color schemes, and so on.

How we'll (usually) use D3

D3 does everything — scales, shapes, DOM manipulation, animation. Most examples you'll find online use all of it together.

A more modern (and in my opinion more intuitive) approach is to use D3 for the math and Svelte for the rendering, often referred to as **declarative** writing.

We'll learn both, starting with the **declarative** method and then comparing to the **imperative** method.

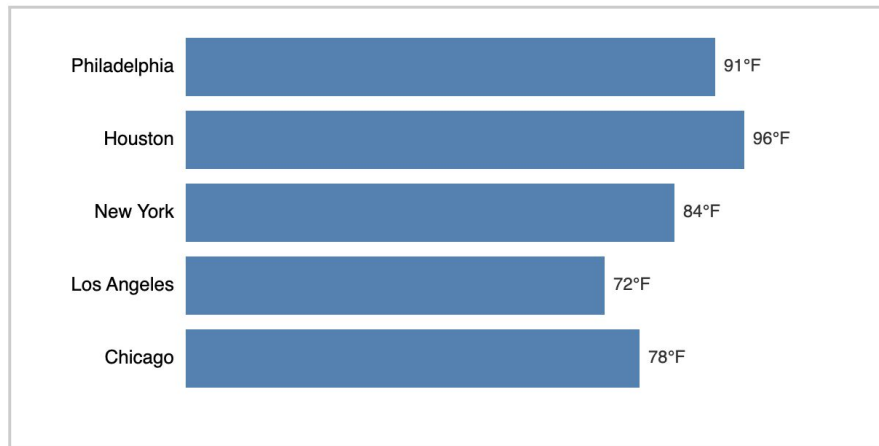
The problem with our scales

Last week, we used hardcoded values:

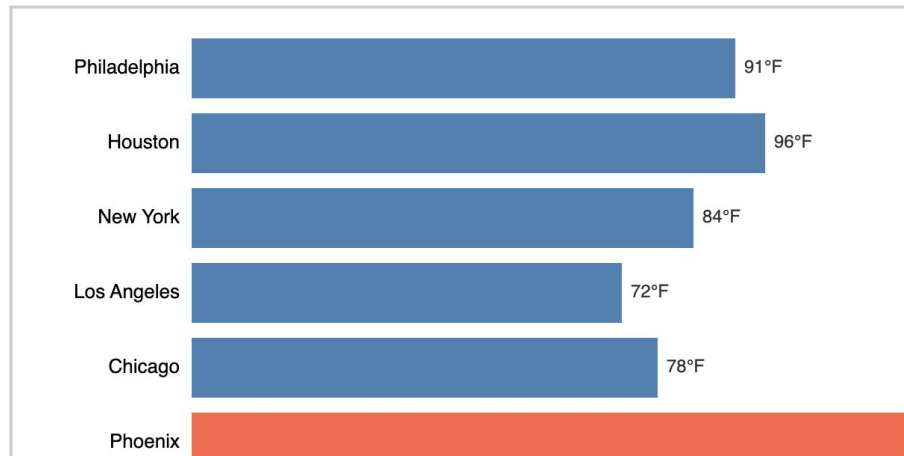
```
width={d.temp}
```

Hardcoded values might work for prototypes, but what happens if

- We add new data?
- The canvas side changes?

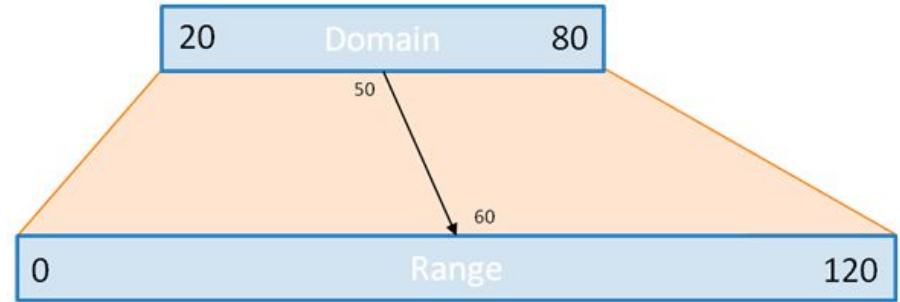


```
const allData = [  
  { city: "Philadelphia", temp: 91 },  
  { city: "Houston",      temp: 96 },  
  { city: "New York",     temp: 84 },  
  { city: "Los Angeles",  temp: 72 },  
  { city: "Chicago",      temp: 78 },  
  { city: "Phoenix",      temp: 310 },  
  { city: "Miami",        temp: 480 },  
  { city: "Dallas",       temp: 850 },  
]
```



What is a scale?

A scale is a function that takes a value from your data's world and returns a value from your visual world.



Source: [Jerome Cukier](#)

data world → **visual world**

\$47,000 → 210px

01/05/2025 → 440px

"Northeast" → steelblue

463 → 0.7

Domain and range

Every D3 scale has two settings:

- **Domain:** the extent of your data. The minimum and maximum values you expect to see.
- **Range:** the extent of your visual output. Usually pixels, sometimes colors.

```
const scale = d3.scaleLinear()  
  .domain([0, 100])    // data goes from 0 to 100  
  .range([0, 500])     // pixels go from 0 to 500  
  
scale(0)    // returns 0  
scale(50)   // returns 250  
scale(100)  // returns 500
```

Working with your data's range

Hard-coding a domain only works if you already know your data's min and max. For real datasets, derive it using

```
const data = [  
  { city: "New York",    temp: 84 },  
  { city: "Philadelphia", temp: 91 },  
  { city: "Chicago",     temp: 78 },  
  { city: "Los Angeles", temp: 72 },  
  { city: "Houston",     temp: 96 }  
]  
  
d3.min(data, d => d.temp) // 72  
d3.max(data, d => d.temp) // 96  
d3.extent(data, d => d.temp) // [72, 96]  
  
// Plug directly into your scale  
const yScale = d3.scaleLinear()  
  .domain([0, d3.max(data, d => d.temp)])  
  .range([height, 0])
```

scaleLinear

For quantitative data — anything you'd measure.

- Temperature
- Income
- Population
- Time

```
const xScale = d3.scaleLinear()  
  .domain([0, 100])  
  .range([0, width])
```

```
const yScale = d3.scaleLinear()  
  .domain([0, 100])  
  .range([height, 0])
```


scaleTime

For temporal data — dates and times. Works like `scaleLinear` but understands the JavaScript `Date` object.

```
const xScale = d3.scaleTime()  
  .domain([new Date("2020-01-01"), new Date("2024-01-01")])  
  .range([0, width])  
  
xScale(new Date("2022-01-01")) // returns a pixel position
```

scaleOrdinal

For mapping categories to colors, shapes, or any other discrete output.

```
const colorScale = d3.scaleOrdinal()  
  .domain(["Northeast", "Midwest", "West", "South"])  
  .range(["#4e79a7", "#f28e2b", "#e15759", "#76b7b2"])
```

scaleBand

For categorical data on an axis — anything you'd list.
Used when you want evenly spaced bars or columns.

- Cities
- Months
- Names
- Categories

```
const xScale = d3.scaleBand()  
  .domain(["Jan", "Feb", "Mar", "Apr"])  
  .range([0, width])  
  .padding(0.2)
```

scaleBand gives you two things:

- `xScale("Jan")` — the x position of that bar
- `xScale.bandwidth()` — the width of each bar
(calculated automatically)



Build Period

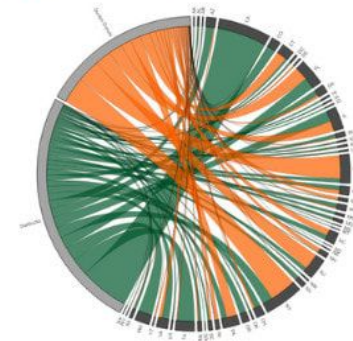
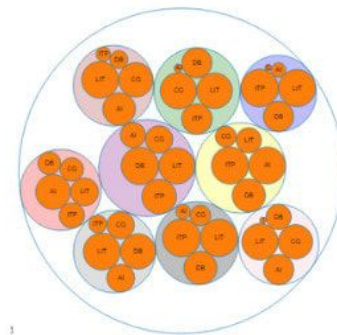
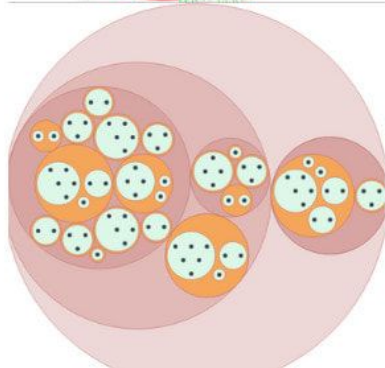
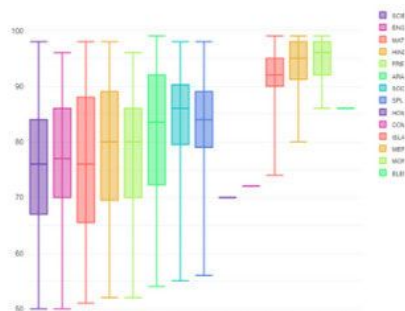
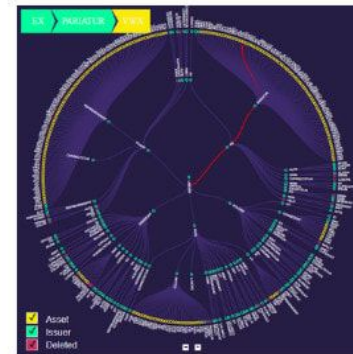
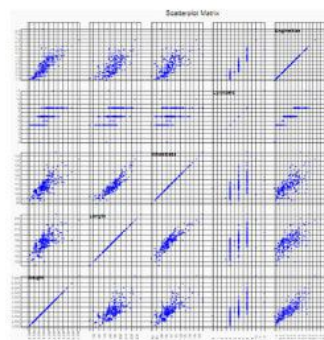
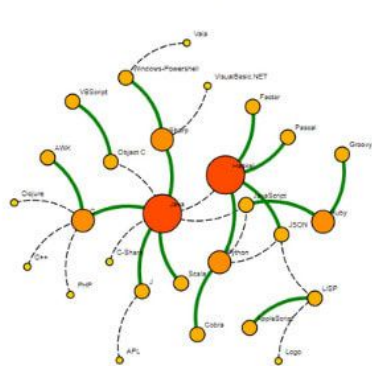
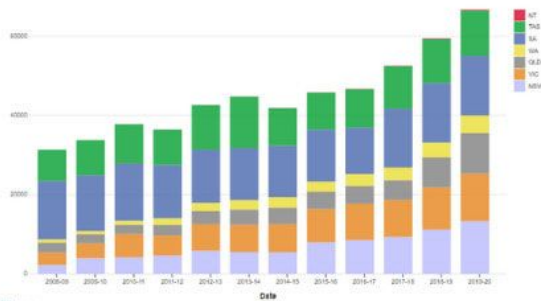
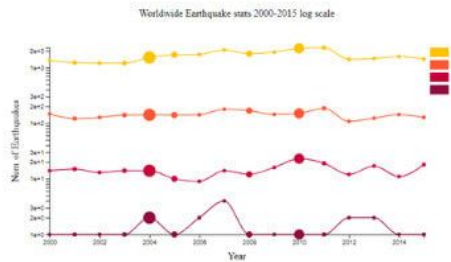


What is a generator?

Last segment, D3 gave us functions that translate data values into pixels. Shape generators do something similar — they take your entire data array and return a single SVG path string.

You've already written paths by hand using `<polyline>`. This is the same idea, but D3 handles the math.

```
const lineGenerator = d3.line()  
  .x(d => xScale(d.month))  
  .y(d => yScale(d.temp))  
  
lineGenerator(data) // "M 60,278 L 160,265 L 260,232..."
```



d3.line()

Takes an array of data points and returns a path string connecting them.

`.x()` and `.y()` are accessor functions. They receive each data point and return the pixel position for that axis — the same logic you've been writing as inline attributes in `{#each}` loops.

```
const lineGenerator = d3.line()  
  .x(d => xScale(d.month))  
  .y(d => yScale(d.temp))  
</script>  
  
<path  
  d={lineGenerator(data)}  
  fill="none"  
  stroke="steelblue"  
  stroke-width="2"  
>
```

d3.area()

Works the same way as `d3.line()` but produces a filled shape between two lines.

`.y0()` is the baseline — usually the bottom of the chart. `.y1()` is the data line. Everything between them gets filled.

```
const areaGenerator = d3.area()  
  .x(d => xScale(d.month))  
  .y0(height - margin.bottom) // baseline  
  .y1(d => yScale(d.temp))    // data line
```


Declarative vs. imperative

Everything you've built so far has been **declarative**. You describe what the output should look like and Svelte renders it.

Most D3 code (or at least older examples) you'll find in the wild is **imperative**. D3 selects DOM elements and builds the chart directly, step by step.

Declarative — describe what you want.

Imperative — describe what to do

Declarative

```
<script>
  import * as d3 from 'd3'

  const width = 400;
  const height = 400;

  const lineGenerator = d3.line()
    .x(d => xScale(d.month))
    .y(d => yScale(d.temp))
</script>

<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>
```

Imperative

```
const svg = d3.select("#chart")
  .append("svg")
  .attr("width", width)
  .attr("height", height)

svg.append("path")
  .datum(data)
  .attr("d", lineGenerator)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2)
```

There are arguments for both, but preference matters

The scales and generators are identical in both approaches. The difference is only in how the output gets rendered.

D3 doesn't care which method you use — it just returns a string or a number. What you do with that value is up to you.



Build Period

